

SQL Query Results

```
3  -- =====
4  -- BEGINNER
5  -- SELECT, WHERE, ORDER BY, LIMIT
6  -- =====
7
8
9  -- 1. List all students from the Graduate education level.
10
11 • SELECT * FROM students
12 WHERE education_level = 'Graduate';
```

Result Grid	Filter Rows:	Edit:	Export/Import:
student_id	student_name	age	education_level
1	Arun	22	Graduate
3	Rahul	21	Graduate
4	Anjali	23	Graduate
6	Sneha	22	Graduate
7	Akhil	25	Graduate
10	Amal	21	Graduate
11	Fathima	24	Graduate
13	Neha	22	Graduate
14	Rohit	25	Graduate
NULL	NULL	NULL	NULL

```
15 -- 2. Show all enrollments made between January and March 2025.
16
17 • SELECT * FROM enrollments
18 WHERE enrollment_date BETWEEN '2025-01-01' AND '2025-03-31';
19
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell C

	enrollment_id	student_id	course_id	enrollment_date	completion_status	score	hours_spent
▶	1	1	101	2025-01-10	Completed	85.00	40.00
	2	1	102	2025-02-15	Completed	91.00	55.00
	3	2	102	2025-01-20	Completed	88.00	50.00
	4	2	103	2025-03-10	Completed	92.00	45.00
	5	3	104	2025-02-05	Completed	78.00	35.00
	6	3	105	2025-03-15	Completed	82.00	30.00
	7	4	102	2025-01-25	Completed	95.00	60.00
	8	4	103	2025-02-20	Completed	89.00	42.00
	9	5	106	2025-01-12	Completed	94.00	70.00
	10	5	102	2025-03-05	Completed	90.00	55.00
	11	6	105	2025-02-10	In Progress	65.00	20.00
	12	6	107	2025-03-20	Completed	84.00	38.00
	13	7	101	2025-01-18	Completed	76.00	32.00
	14	7	104	2025-02-25	Completed	81.00	36.00

students 1

enrollments 2 ×

enrollments 3

enrollments 4

students 5

courses 6

courses 7

22

23 • SELECT *

24 FROM enrollments

```
25 ORDER BY score DESC
```

```
26     LIMIT 10;
```

27

[illegible]

```
29 -- 4. List all enrollments with status = 'Completed'.
```

30

```
31 • SELECT * FROM enrollments
```

```
32 WHERE completion_status = 'Completed';
```

33

Result Grid							
Filter Rows: Edit: Export/Import: Wrap Cell							
	enrollment_id	student_id	course_id	enrollment_date	completion_status	score	hours_spent
1	1	1	101	2025-01-10	Completed	85.00	40.00
2	1	1	102	2025-02-15	Completed	91.00	55.00
3	2	2	102	2025-01-20	Completed	88.00	50.00
4	2	2	103	2025-03-10	Completed	92.00	45.00
5	3	3	104	2025-02-05	Completed	78.00	35.00
6	3	3	105	2025-03-15	Completed	82.00	30.00
7	4	4	102	2025-01-25	Completed	95.00	60.00
8	4	4	103	2025-02-20	Completed	89.00	42.00
9	5	5	106	2025-01-12	Completed	94.00	70.00
10	5	5	102	2025-03-05	Completed	90.00	55.00
12	6	6	107	2025-03-20	Completed	84.00	38.00
13	7	7	101	2025-01-18	Completed	76.00	32.00
14	7	7	104	2025-02-25	Completed	81.00	36.00
15	8	8	102	2025-01-30	Completed	93.00	58.00

```

35  -- 5. Find students who are above 23 years old.
36
37 • SELECT * FROM students
38   WHERE age > 23;
39

```

Result Grid				
		Filter Rows:		
		Edit:		
		Export/Import:		
	student_id	student_name	age	education_level
▶	2	Meera	24	Post Graduate
	5	Vishnu	26	Post Graduate
	7	Akhil	25	Graduate
	9	Nikhil	27	Post Graduate
	11	Fathima	24	Graduate
	12	Adithya	26	Post Graduate
	14	Rohit	25	Graduate
*	NULL	NULL	NULL	NULL

```

41  -- 6. Find the 5 most expensive courses.
42
43 • SELECT * FROM courses
44   ORDER BY course_fee DESC
45   LIMIT 5;
46

```

Result Grid					
		Filter Rows:			
		Edit:			
		Export/Import:			
	course_id	course_name	category	instructor	course_fee
▶	106	Machine Learning Basics	Data Science	Kevin Thomas	7500.00
	102	Data Analytics	Data Analytics	Sarah Thomas	6500.00
	108	Advanced Python	Programming	John Mathew	6000.00
	107	Digital Marketing	Marketing	Priya Nair	5500.00
	103	Power BI Essentials	Data Analytics	David Paul	5000.00
*	NULL	NULL	NULL	NULL	NULL

```

48  -- 7. List all courses belonging to the Data Analytics category.
49
50 • SELECT * FROM courses
51   WHERE category = 'Data Analytics';
52

```

Result Grid					
		Filter Rows:			
		Edit:			
		Export/Import:			
	course_id	course_name	category	instructor	course_fee
▶	102	Data Analytics	Data Analytics	Sarah Thomas	6500.00
	103	Power BI Essentials	Data Analytics	David Paul	5000.00
*	NULL	NULL	NULL	NULL	NULL

```

3  -- =====
4  -- INTERMEDIATE
5  -- JOINS, GROUP BY, HAVING, AGGREGATES
6  -- =====
7
8
9  -- 8. What is the total number of enrollments?
10
11 • SELECT COUNT(*) AS total_enrollments
12    FROM enrollments;
13

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	total_enrollments				
▶	30				

```

15  -- 9. Which course has the highest number of enrollments?
16
17 • SELECT c.course_name,
18        COUNT(e.enrollment_id) AS total_enrollments
19    FROM enrollments as e
20   JOIN courses as c
21  ON e.course_id = c.course_id
22   GROUP BY c.course_id, c.course_name
23   ORDER BY total_enrollments DESC
24   LIMIT 1;
25

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	course_name	total_enrollments			
▶	Data Analytics	7			

```

27  -- 10. List the top 5 students by total learning hours.
28
29 • SELECT s.student_name,
30        SUM(e.hours_spent) AS total_hours
31    FROM enrollments e
32   JOIN students s
33  ON e.student_id = s.student_id
34   GROUP BY s.student_id, s.student_name
35   ORDER BY total_hours DESC
36   LIMIT 5;
37

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	student_name	total_hours			
▶	Adithya	130.00			
	Vishnu	125.00			
	Diya	123.00			
	Nikhil	118.00			
	Aparna	103.00			

```

39  -- 11. Which courses have more than 3 enrolled students?
40
41  • SELECT c.course_name,
42         COUNT(e.student_id) AS total_students
43  FROM enrollments e
44  JOIN courses c
45  ON e.course_id = c.course_id
46  GROUP BY c.course_id, c.course_name
47  HAVING total_students > 3;
48

```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	course_name	total_students
▶	Data Analytics	7
	Power BI Essentials	4
	SQL Fundamentals	4
	Excel for Business	4
	Machine Learning Basics	4

```

50  -- 12. What is the average score for each education level?
51
52  • SELECT s.education_level,
53         AVG(e.score) AS average_score
54  FROM enrollments e
55  JOIN students s
56  ON e.student_id = s.student_id
57  GROUP BY s.education_level;
58

```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	education_level	average_score
▶	Graduate	81.777778
	Post Graduate	91.250000

```

60  -- 13. What is the average score for each course?
61
62  • SELECT c.course_name,
63         AVG(e.score) AS average_score
64  FROM enrollments e
65  JOIN courses c
66  ON e.course_id = c.course_id
67  GROUP BY c.course_id, c.course_name;
68

```

Result Grid   Filter Rows: | Export:  | Wrap Cell C

	course_name	average_score
▶	Python for Beginners	73.666667
	Data Analytics	91.428571
	Power BI Essentials	90.000000
	SQL Fundamentals	81.000000
	Excel for Business	77.750000
	Machine Learning Basics	92.000000
	Digital Marketing	82.000000
	Advanced Python	89.500000

```

70 -- 14. How many completed students are there for each course,
71 -- and what is the average score?
72
73 • SELECT c.course_name,
74      COUNT(CASE
75          WHEN e.completion_status = 'Completed' THEN 1
76          END) AS completed_students,
77      AVG(e.score) AS average_score
78 FROM enrollments e
79 JOIN courses c
80 ON e.course_id = c.course_id
81 GROUP BY c.course_id, c.course_name;
82

```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	course_name	completed_students	average_score
▶	Python for Beginners	2	73.666667
	Data Analytics	7	91.428571
	Power BI Essentials	4	90.000000
	SQL Fundamentals	4	81.000000
	Excel for Business	3	77.750000
	Machine Learning Basics	4	92.000000
	Digital Marketing	2	82.000000
	Advanced Python	2	89.500000

```

84 -- 15. Which course categories have more than 5 enrollments?
85
86 • SELECT c.category,
87      COUNT(e.enrollment_id) AS total_enrollments
88 FROM enrollments e
89 JOIN courses c
90 ON e.course_id = c.course_id
91 GROUP BY c.category
92 HAVING total_enrollments > 5;
93

```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	category	total_enrollments
▶	Data Analytics	11

```

95      -- 16. What is the total learning time for each course?
96
97      • SELECT c.course_name,
98             SUM(e.hours_spent) AS total_hours
99      FROM enrollments e
100     JOIN courses c
101     ON e.course_id = c.course_id
102     GROUP BY c.course_id, c.course_name;
103

```

Result Grid   Filter Rows: Export:  Wrap Cell Content:

	course_name	total_hours
▶	Python for Beginners	90.00
	Data Analytics	387.00
	Power BI Essentials	177.00
	SQL Fundamentals	142.00
	Excel for Business	111.00
	Machine Learning Basics	275.00
	Digital Marketing	73.00
	Advanced Python	108.00

```

105      -- 17. Find the highest score obtained in each course.
106
107      • SELECT c.course_name,
108             MAX(e.score) AS highest_score
109      FROM enrollments e
110     JOIN courses c
111     ON e.course_id = c.course_id
112     GROUP BY c.course_id, c.course_name;

```

Result Grid   Filter Rows: Export:  Wrap Cell Content:

	course_name	highest_score
▶	Python for Beginners	85.00
	Data Analytics	95.00
	Power BI Essentials	92.00
	SQL Fundamentals	90.00
	Excel for Business	85.00
	Machine Learning Basics	96.00
	Digital Marketing	84.00
	Advanced Python	93.00

```

4      -- ADVANCED
5      -- SUBQUERIES, CTEs, WINDOW FUNCTIONS, VIEWS, STORED PROCEDURES
6      -- =====
7
8
9      -- 18. Find students who scored higher than the overall
10     --      average score.
11
12     •  SELECT s.student_name, e.score
13         FROM students s
14         JOIN enrollments e
15         ON s.student_id = e.student_id
16         WHERE e.score > (
17             SELECT AVG(score) FROM enrollments
18         )
19     ORDER BY e.score DESC;

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	student_name	score			
▶	Adithya	96.00			
	Anjali	95.00			
	Vishnu	94.00			
	Aparna	94.00			
	Diya	93.00			
	Adithya	93.00			
	Meera	92.00			
	Arun	91.00			
	Nikhil	91.00			
	Aparna	91.00			
	Vishnu	90.00			
	...	...			

Result 1 × Result 2 students 3 Result 4 Result 5 course_performance_summary 6

```

22  -- 19. Find students who spent more hours than the
23  --      average learning hours.
24
25  •  SELECT s.student_name, e.hours_spent
26      FROM students s
27      JOIN enrollments e
28      ON s.student_id = e.student_id
29      WHERE e.hours_spent > (
30          SELECT AVG(hours_spent)
31          FROM enrollments
32      )
33      ORDER BY e.hours_spent DESC;
34

```

Result Grid			Filter Rows:	Export:	Wrap Cell Con
	student_name	hours_spent			
▶	Adithya	72.00			
	Vishnu	70.00			
	Nikhil	68.00			
	Diya	65.00			
	Anjali	60.00			
	Diya	58.00			
	Adithya	58.00			
	Aparna	57.00			
	Arun	55.00			
	Vishnu	55.00			
	Neha	52.00			
	..	52.00			

Result 1 Result 2 × students 3 Result 4 Result 5 course_perf

```

36  -- 20. Identify students who enrolled in more than one course.
37
38  • SELECT s.student_name
39      FROM students s
40  WHERE s.student_id IN (
41      SELECT student_id
42      FROM enrollments
43      GROUP BY student_id
44      HAVING COUNT(course_id) > 1
45  );

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
student_name			
▶ Arun			
Meera			
Rahul			
Anjali			
Vishnu			
Sneha			
Akhil			
Diya			
Nikhil			
Amal			
Fathima			

Result 1 Result 2 students 3 × Result 4 Result 5 course_performance_summ

```

48  -- 21. Rank students based on their score.
49
50  • SELECT s.student_name, e.score,
51      RANK() OVER (
52          ORDER BY e.score DESC
53      ) AS score_rank
54  FROM students s
55  JOIN enrollments e
56  ON s.student_id = e.student_id;
57

```

Result Grid	Filter Rows:	Export:
student_name	score	score_rank
▶ Adithya	96.00	1
Anjali	95.00	2
Vishnu	94.00	3
Aparna	94.00	3
Diya	93.00	5
Adithya	93.00	5
Meera	92.00	7
Arun	91.00	8
Nikhil	91.00	8
Aparna	91.00	8
Vishnu	90.00	11

Result 1 Result 2 students 3 Result 4 × Result 5

```

59  -- 22. Create a CTE showing course-wise enrollment
60  --      and average score.
61
62  • WITH course_analysis AS (
63      SELECT c.course_name,
64             COUNT(e.enrollment_id) AS total_enrollments,
65             AVG(e.score) AS average_score
66      FROM enrollments e
67      JOIN courses c
68      ON e.course_id = c.course_id
69      GROUP BY c.course_id, c.course_name
70  )
71
72  SELECT *
73  FROM course_analysis
74  ORDER BY average_score DESC;
75

```

Result Grid			
Filter Rows:		Export:	Wrap Cell Content:
course_name	total_enrollments	average_score	
Machine Learning Basics	4	92.000000	
Data Analytics	7	91.428571	
Power BI Essentials	4	90.000000	
Advanced Python	2	89.500000	
Digital Marketing	2	82.000000	
SQL Fundamentals	4	81.000000	
Excel for Business	4	77.750000	
Python for Beginners	3	73.666667	

Result 1
Result 2
students 3
Result 4
Result 5 ×
course_performan

```

77 -- 23. Create a view called course_performance_summary.
78
79 • CREATE VIEW course_performance_summary AS
80 SELECT c.course_name,
81        c.category,
82        COUNT(e.enrollment_id) AS total_enrollments,
83        AVG(e.score) AS average_score,
84        SUM(e.hours_spent) AS total_hours
85 FROM courses c
86 JOIN enrollments e
87 ON c.course_id = e.course_id
88 GROUP BY c.course_id, c.course_name, c.category;
89
90
91 -- View the result
92
93 • SELECT *
94 FROM course_performance_summary;
95

```

Result Grid					
		Filter Rows:	Export:		Wrap Cell Content:
	course_name	category	total_enrollments	average_score	total_hours
▶	Python for Beginners	Programming	3	73.666667	90.00
	Data Analytics	Data Analytics	7	91.428571	387.00
	Power BI Essentials	Data Analytics	4	90.000000	177.00
	SQL Fundamentals	Database	4	81.000000	142.00
	Excel for Business	Business	4	77.750000	111.00
	Machine Learning Basics	Data Science	4	92.000000	275.00
	Digital Marketing	Marketing	2	82.000000	73.00
	Advanced Python	Programming	2	89.500000	108.00

Result 1 Result 2 students 3 Result 4 Result 5 **course_performance_summary 6** ×

```

97      -- 24. Rank students within each course.
98
99      • SELECT c.course_name,
100             s.student_name,
101             e.score,
102             RANK() OVER (
103                 PARTITION BY c.course_id
104                 ORDER BY e.score DESC
105             ) AS course_rank
106 FROM courses c
107 JOIN enrollments e
108 ON e.course_id = c.course_id
109 JOIN students s
110 ON s.student_id = e.student_id
111 ORDER BY c.course_name, course_rank;
112
113

```

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
	course_name	student_name	score	course_rank			
▶	Advanced Python	Adithya	93.00	1			
	Advanced Python	Nikhil	86.00	2			
	Data Analytics	Anjali	95.00	1			
	Data Analytics	Aparna	94.00	2			
	Data Analytics	Diya	93.00	3			
	Data Analytics	Arun	91.00	4			
	Data Analytics	Vishnu	90.00	5			
	Data Analytics	Neha	89.00	6			
	Data Analytics	Meera	88.00	7			

Result 1 Result 2 students 3 Result 4 Result 5 course_performance_summary 6 [Result 7](#) ×

```

114 -- 25. Create a stored procedure to show a student's
115 --     complete course history.
116
117 DELIMITER //
118 • CREATE PROCEDURE get_student_history(IN p_student_id INT)
119 BEGIN
120     SELECT s.student_id,
121            s.student_name,
122            c.course_name,
123            c.category,
124            e.enrollment_date,
125            e.completion_status,
126            e.score,
127            e.hours_spent
128 FROM students s
129 JOIN enrollments e
130 ON s.student_id = e.student_id
131 JOIN courses c
132 ON e.course_id = c.course_id
133 WHERE s.student_id = p_student_id
134 ORDER BY e.enrollment_date;
135 END //
136 DELIMITER ;

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 								
	student_id	student_name	course_name	category	enrollment_date	completion_status	score	hours_spent
▶	1	Arun	Python for Beginners	Programming	2025-01-10	Completed	85.00	40.00
	1	Arun	Data Analytics	Data Analytics	2025-02-15	Completed	91.00	55.00